

Lab 5: Pre lab - Synthesizable vs Non-Synthesizable Verilog

Pre-Lab Concept: Synthesizable vs Non-Synthesizable Verilog

Key Concept

Verilog can be used for **two very different purposes**:

- **Simulation:** To model and test behavior
- **Synthesis:** To generate real hardware on an FPGA

Not all Verilog constructs that work in simulation can be translated into physical hardware.

Key idea: *If you cannot imagine a piece of hardware implementing it, it is probably not synthesizable.*

1. Loops in Verilog: Simulation vs Synthesis

Example

Synthesizable for Loop

- Loop bounds are constant
- Loop represents replicated hardware

```
integer i;
always @(*) begin
    for (i = 0; i < 8; i = i + 1)
        y[i] = a[i] & b[i];
end
```

Hardware meaning: 8 parallel AND gates inferred by the synthesizer.

Common Pitfall

Non-Synthesizable Loop

```
always @(*) begin
    i = 0;
    while (a[i] != 1'b1)
        i = i + 1;
end
```

Why this fails synthesis:

- Unknown iteration count
- Hardware size cannot change dynamically

2. Timing Controls and Delays

Common Pitfall

Simulation-Only Delay

```
always begin
  clk = 0;
  #5 clk = 1;
  #5;
end
```

Arbitrary delays are not supported in FPGA hardware.

Example

Synthesizable Alternative

```
always @(posedge clk) begin
  q <= d;
end
```

Clock timing is provided by external oscillators or FPGA clocking resources.

3. Reset and Initialization

Common Pitfall

Simulation-Only Initialization

```
initial begin
  q = 0;
end
```

initial blocks are unreliable for sequential logic in synthesis.

Example

Synthesizable Reset

```
always @(posedge clk) begin
  if (reset)
    q <= 0;
  else
    q <= d;
end
```

Reset logic ensures a known hardware startup state.

4. Continuous Assignment vs Procedural Assignment

Example

Continuous Assignment

```
assign y = a & b;
```

Pure combinational logic.

Common Pitfall

Multiple Drivers (Illegal)

```
assign y = a & b;
```

```
always @(*) begin
```

```
    y = a | b;
```

```
end
```

Two hardware drivers for one signal cause synthesis failure.

Exercises (Pre-Lab)

Exercise

Exercise 1: Identify Synthesizability

```
always @(*) begin
```

```
    for (i = 0; i < n; i = i + 1)
```

```
        sum = sum + a[i];
```

```
end
```

Exercise

Exercise 2: Fix the Code

```
always begin
```

```
    #10 clk = ~clk;
```

```
end
```

Exercise

Exercise 3: What hardware does this code describe?

```
always @(posedge clk) begin
```

```
    if (en)
```

```
        q <= d;
```

```
end
```

From RTL to FPGA: Bitstream Generation

FPGA Design Flow

- (a) **Synthesis:** RTL $\rightarrow$ FPGA primitives (LUTs, FFs)
- (b) **Implementation (Place & Route)**
- (c) **Timing Analysis**
- (d) **Bitstream Generation**
- (e) **FPGA Programming via USB/JTAG**

Key Concept

Simulation verifies functional correctness. Only synthesis and implementation guarantee reliable FPGA operation.

Lab 5: Designing FSM Using FPGA Switches and LEDs

Objective

The objective of this lab is to design, implement, and verify a Finite State Machine (FSM) on an FPGA using input switches, LEDs, and a reset button. Students will learn how to interface user inputs with hardware logic, implement state-based control, and manage synchronous counters using Verilog HDL.

By the end of this lab, students will be able to:

- Design an FSM using a state diagram
- Interface FPGA switches and LEDs with control logic
- Implement a decrement counter controlled by FSM states
- Integrate provided switch, button debouncing, and LED interface modules
- Verify FSM behavior through FPGA synthesis and on-board testing

Finite State Machine FSM

A Finite State Machine is a sequential circuit whose output depends on the current state and, depending on the design, the current inputs. FSMs are widely used in digital systems to control step-by-step behavior such as waiting for input, loading data, counting, resetting, and returning to an idle state.

In this lab, the FSM provides a simple hardware-control example before students begin designing processor modules. The same design ideas appear later in processor control logic, where instruction decoding, branch decisions, memory access, and write-back control must be coordinated correctly. Therefore, this lab should be treated as preparation for understanding processor control, not as an isolated FPGA exercise.

System Description

The system behavior is defined as follows:

- The system continuously monitors the FPGA switches for input.
- If the switch input value is **zero**, the system remains in the input-waiting state.
- When the switch input becomes **non-zero**, the FSM:
 - Captures the switch value
 - Displays the value on the LEDs
 - Starts a decrement counter from the captured value down to zero
- While counting down:
 - The LEDs reflect the current counter value
 - Switch inputs are ignored
- When the counter reaches zero:

- The FSM automatically returns to the input-waiting state
- At any time during counting, pressing the **reset button**:
 - Immediately returns the FSM to the input-waiting state
 - Clears the counter and LED output

Note: The switch interface, button debouncing interface, and LED interface are provided and must be used without modification.

Task 1: FSM Design and State Diagram

Objective

To design an FSM that controls system behavior based on switch input, counter value, and reset signal.

Procedure

- (a) Identify the required FSM states, such as:
 - Input Waiting State
 - Countdown State
 - Reset State (if implemented explicitly)
- (b) Draw a complete FSM state diagram showing:
 - State transitions
 - Conditions for transitions (switch input, counter reaching zero, reset)
 - Outputs associated with each state
- (c) Clearly indicate:
 - How switch input is latched
 - When the counter is enabled or disabled
 - How LEDs are driven in each state

Deliverables

- FSM state diagram with labeled transitions and states

Task 2: Verilog FSM and Counter Implementation

Objective

To implement the FSM and decrement counter in Verilog HDL and verify functionality through simulation.

Procedure

(a) Implement the FSM in Verilog:

- Use synchronous state transitions triggered by the system clock
- Include an asynchronous or synchronous reset as specified

(b) Implement a decrement counter:

- Load the counter with the switch input value
- Decrement the counter on each clock cycle while enabled
- Stop counting when the counter reaches zero

(c) Integrate the provided modules:

- Switch interface:

```
module switches(  
    input clk,  
    input rst,  
    input [31:0] writeData,  
    input writeEnable,  
    input readEnable,  
    input [29:0] memAddress,  
  
    output reg [31:0] readData = 0, // not to be read  
    output reg [15:0] leds  
);  
...  
endmodule
```

- Button debouncing interface:

```
module debouncer(  
    input clk,  
    input pbin,  
    output pbout
```

- LED interface:

```
module leds(  
    input clk, rst,  
    input [15:0] btns,  
    input [31:0] writeData, // not to be written  
    input writeEnable, // not to be used  
    input readEnable,  
    input [29:0] memAddress,  
    input [15:0] switches,
```

```
output reg [31:0] readData
```

(d) Develop a testbench to:

- Apply different switch input values
- Verify FSM transitions
- Confirm correct countdown behavior
- Verify reset functionality

Deliverables

- Verilog source file for FSM and counter
- Testbench file
- Simulation waveforms showing correct operation

Task 3: FPGA Constraints, XDC File and Timing Analysis

Objective

To introduce students to FPGA pin constraints, clock constraints, and timing analysis using Vivado synthesis reports.

Background

FPGA designs require a constraint file (XDC – Xilinx Design Constraints) to define:

- Which physical FPGA pins connect to design signals
- What clock frequency the design must meet
- Input/output timing requirements

Without a correct XDC file, synthesis cannot guarantee correct operation.

XDC Constraint File

Each top-level FPGA signal must be mapped to a physical FPGA pin.

Example Pin Constraint (Switch)

Map Switch Input to FPGA Pin

```
set_property PACKAGE_PIN V17 [get_ports {switches[0]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {switches[0]}]
```

Explanation:

- PACKAGE_PIN V17 specifies the physical FPGA pin
- IOSTANDARD LVCMOS33 defines voltage level (3.3V CMOS)

Example Pin Constraint (LED)**Map LED Output to FPGA Pin**

```
set_property PACKAGE_PIN U16 [get_ports {leds[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {leds[0]}]
```

Clock Constraint**Define System Clock (100 MHz)**

```
create_clock -period 10.000 -name sys_clk [get_ports clk]
```

Explanation: The clock period of 10 ns corresponds to 100 MHz. All logic must complete within this time.

Understanding Timing Slack

$$\text{Slack} = \text{Required Time} - \text{Actual Path Delay}$$

Slack Value	Meaning	Status
Positive	Timing met	PASS
Zero	Critical timing	WARNING
Negative	Timing violation	FAIL

Real Timing Report Examples**Example 1 – Good Design (PASS)**

```
Slack (MET) : 3.215ns
Required Time : 10.000ns
Path Delay : 6.785ns
```

Interpretation: Design runs safely at 100 MHz.

Example 2 – Marginal Design (CRITICAL)

```
Slack (MET) : 0.042ns
Required Time : 10.000ns
Path Delay : 9.958ns
```

Interpretation: Design barely meets timing and may fail under temperature/voltage variation.

Example 3 – Failing Design (VIOLATION)

Slack (VIOLATED) : -1.372ns
Required Time : 10.000ns
Path Delay : 11.372ns

Interpretation: Design cannot operate at 100 MHz — must be optimized or clock slowed.

Procedure

- (a) Add an XDC file to your project
- (b) Assign pin constraints to all switches, LEDs, reset, and clock
- (c) Add a clock constraint
- (d) Run synthesis
- (e) Report your Worst Negative Slack (WNS)

Deliverables

- XDC file
- Screenshot of Vivado Timing Summary
- WNS value and interpretation (PASS / MARGINAL / FAIL)

Task 4: FPGA Synthesis and On-Board Verification

Objective

To synthesize the FSM design on an FPGA and verify functionality using physical switches, LEDs, and reset button.

Procedure

- (a) Integrate the FSM module into the top-level FPGA design.
- (b) Assign FPGA pins for:
 - Switch inputs
 - LED outputs
 - Reset button
- (c) Synthesize the design and analyze:
 - RTL schematic

- State transitions
 - Counter implementation
- (d) Program the FPGA and perform on-board testing:
- Apply zero and non-zero switch values
 - Observe LED countdown behavior
 - Press reset during countdown to verify immediate reset
- (e) Demonstrate correct operation to the lab instructor or RA.

Deliverables

- RTL schematic screenshots
- Successful FPGA demonstration

Expected Learning Outcomes

Upon successful completion of this lab, students will be able to:

- Design FSMs for hardware control applications
- Interface user inputs with synchronous digital systems
- Implement counters controlled by FSM logic
- Debug and verify FPGA-based designs using real hardware



Assessment Rubric

Lab 5: Designing FSM Using FPGA Switches and LEDs

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Pre lab	-	/15	-
	Task 1	-	-	/10
	Task 2	/20	/20	-
	Task 3	-	/15	-
Total Points		/20	/50	/10
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided [here](#).